Federated Biomedical Knowledge Graphs at Scale: Graph-Native Storage, Cross-KG Queries, and AI-Agent Integration

Madhulatha Mandarapu[1] and Sandeep Kunkunuru[1]

VaidhyaMegha Private Limited, Hyderabad, India
{madhulatha,sandeep}@samyama.ai

Abstract. Answering biomedical questions that cross database boundaries—such as linking adverse-event reports to the biological pathways of drug targets in ongoing clinical trials—typically requires bespoke scripting across siloed data sources. We report on a two-year effort to federate ten public biomedical knowledge graphs (137 million nodes, 1.22 billion edges) inside a single graph database instance and evaluate the resulting system on 500 cross-domain queries.

Three findings emerged. First, *bridge-property federation*—matching shared identifiers (genes, drugs, country codes) across independently loaded KGs in standard OpenCypher—scales to billion-edge graphs when the query planner maintains exact per-triple-pattern statistics rather than sampled estimates; no schema-alignment middleware is required at query time, though identifier normalization during ETL is essential. Second, a planner optimization we call *adjacency-aware aggregation*, which reads neighbor counts directly from adjacency-list metadata instead of scanning edges, converts the most common class of timeout queries—"count neighbors of type X"—from infeasible to sub-second on graphs with 10^9 edges. Third, exposing each KG to LLM agents through domain-specific Model Context Protocol (MCP) tool definitions achieves 98% accuracy on 40 biomedical questions, compared to 85% for generic text-to-Cypher and 75% for standalone GPT-4o; the accuracy gap is attributable to schema knowledge encoded in the templates, not to model capability. The system, Samyama, achieves 500/500 on the benchmark (468 full passes, 32 documented corpus-coverage gaps) and runs on a single AWS spot instance. We discuss the design trade-offs, limitations, and failure modes encountered during development and evaluation.

Keywords: Knowledge graphs · Biomedical data integration · Graph databases · OpenCypher · Federation · Model Context Protocol

1 Introduction

Biomedical knowledge is fragmented across dozens of public databases, each covering a narrow slice of the translational research pipeline: protein–protein

interactions (STRING [1]), biological pathways (Reactome [2]), drug–gene interactions (DGIdb [3]), side effects (SIDER [4]), bioactivity assays (ChEMBL [5]), clinical trials (ClinicalTrials.gov), post-market adverse events (FDA FAERS), patient-level observational data (OMOP [6]), protein sequences (UniProt [7]), and population-health indicators (WHO, World Bank).

Answering a question such as *"For drugs in Phase 3 breast cancer trials, what are their known adverse events in FAERS and which biological pathways do their targets participate in?"* today requires joining across at least four databases with ad-hoc scripting, identifier reconciliation, and no transactional guarantees.

This paper reports on the design, deployment, and evaluation of Samyama, a graph database for federated biomedical knowledge graph analytics in active use within our research group. Our contributions are empirical findings from building and operating this system:

1. **Bridge-property federation scales to 10^9 edges** (Sections 3–4). Matching shared identifiers across ten independently loaded KGs in standard OpenCypher is practical at billion-edge scale, provided that (a) identifiers are normalized to canonical vocabularies during ETL, and (b) the query planner uses exact per-triple-pattern statistics. We quantify bridge-join coverage rates and document where identifier mismatches cause silent data loss.
2. **Adjacency-aware aggregation eliminates a class of timeout queries** (Section 2). Reading neighbor counts from adjacency-list metadata—rather than scanning edges—converts "count neighbors of type X" queries from infeasible to sub-second on graphs with 10^9 edges.
3. **Transparent benchmarking requires separating engine capability from corpus coverage** (Section 4). Of 500 benchmark queries, 32 return zero rows due to documented data gaps, not engine failures. We describe the oracle methodology and argue that benchmarks that conflate the two overstate capability.
4. **Domain-specific MCP tools outperform generic text-to-Cypher by encoding schema knowledge** (Section 5). On 40 biomedical questions, parameterized Cypher templates achieve 98% accuracy vs. 85% for LLM-generated Cypher—but at the cost of manual authoring effort that does not generalize to unseen query types.

2 System Architecture

Samyama is implemented in Rust (1,933 unit tests, 87.8% line coverage). Rather than cataloging features, we focus on four design decisions that directly influenced the benchmark results in Section 4, discussing the trade-offs each entails.

2.1 Late Materialization

The alternative to cloning full node/edge objects through the operator pipeline is to defer property access until the `RETURN` clause. Samyama takes this approach: scan operators produce 8-byte `NodeRef(id)` values; properties are resolved on demand via $O(1)$ hash-map lookup; the `ExpandOperator` returns

Table 1: Late-materialization micro-benchmark on a 10K-node, 50K-edge graph (Apple M4, 16 GB RAM, single-threaded). Each Cypher row is mean of 100 iterations.

Workload	Eager	Lazy	Speedup
Raw 3-hop traversal (per query)	27.28 μs	22.50 μs	1.21×
Cypher `RETURN n` vs `n.name` (1000 nodes)	2.78 ms	2.23 ms	1.25×

`(EdgeId, NodeId, NodeId, &EdgeType)` tuples without copying edge properties.

Measured impact. We quantify the late-materialization benefit on a synthetic 10K-node, 50K-edge graph (5 outgoing edges per node, three properties per node) using a Criterion micro-benchmark (Table 1). Two contrasts are informative. At the storage API level, traversing edges via the lightweight `get_outgoing_edge_targets` (which returns `(EdgeId, NodeId, NodeId, &EdgeType)` tuples) is 17.5% faster than `get_outgoing_edges` (which clones full `Edge` objects). At the Cypher level, returning a single property (`RETURN n.name`) is 20% faster than returning the full node (`RETURN n`, which forces materialization of all properties at the project step).

Trade-off. The cost of late materialization is an additional indirection per property access in `WHERE` filters—negligible for selective queries but measurable on full-scan aggregations where every node's properties are accessed regardless. For `LIMIT` k queries the benefit is unambiguous: only k nodes are ever fully materialized. Quantifying the peak-memory benefit (rather than just latency) requires an eager-materialization mode that the engine does not currently expose; we leave this to future work.

2.2 Graph-Native Planner

The planner maintains a *GraphCatalog* with triple-level statistics (`:Label`, `:TYPE`, `:Label`) $\rightarrow$ (count, avg_out_degree, avg_in_degree), updated incrementally on edge create/delete. Given a `MATCH` pattern, the plan enumerator generates candidate plans starting from each node variable, choosing traversal direction by comparing avg_out_degree vs. avg_in_degree for each edge pattern.

When both endpoints of an edge are already bound, the enumerator inserts an `ExpandInto` operator that checks edge existence via sorted-adjacency binary search in $O(\log d_{\min})$ instead of a full neighbor scan. A multiplicative cost model scores each candidate:

$$C = \prod_{op \in \text{plan}} f(op)$$

where $f(\text{LabelScan}) = |\text{label}|$, $f(\text{Expand}) = \bar{d}$, $f(\text{ExpandInto}) = P(\text{edge exists})$, $f(\text{Filter}) = \sigma$. The lowest-cost plan is passed through a logical optimizer (predicate pushdown, join reordering) and then translated to physical operators.

2.3 MVCC Transaction Isolation

A practical concern in federated KG deployment is that individual KGs are updated at different cadences—FAERS quarterly, PubMed daily, Clinical Trials continuously. Reloading one KG while analytical queries run on others requires snapshot isolation: readers must see a consistent graph state without blocking writers.

Samyama implements MVCC with snapshot isolation (SI) as the default. Each write creates versioned copies of affected nodes and edges; readers operate on a frozen snapshot. A background garbage collector reclaims stale versions. We chose SI over serializable isolation as the default because our workload is overwhelmingly read-heavy—KG updates are batch imports, not concurrent transactions—and SI avoids the false-positive abort rate of serializable checking on long-running analytical queries.

2.4 Adjacency-Aware Aggregation

During benchmark development, we identified a recurring query pattern that timed out on billion-edge KGs: *"for each node of label X, count its neighbors of type Y."* Examples include counting adverse events per drug (FAERS, 89.9M edges), trials referencing each PubMed article (PubMed, 1.04B edges), and gene targets per pathway (Pathways, 835K edges). The naïve plan—expand all edges into the pipeline, group by source node, count—is $O(|E|)$ and exceeded our 120-second timeout on PubMed.

The key insight is that the answer is already encoded in the adjacency-list metadata: each node stores a per-edge-type neighbor count as part of its adjacency header. We added an `AdjacencyCountAggregate` logical plan node that the planner substitutes when it detects the `MATCH (a)-[:T]->(b) RETURN a, count(b)` shape, reading the count in $O(1)$ per source node without materializing any edges.

Impact. Seven benchmark queries moved from timeout (>120 s) to sub-second (0.2–0.8 s), accounting for the largest single improvement in the benchmark results (Section 4).

Limitation. The optimization applies only to unfiltered neighbor counts. Queries like "count neighbors where `b.property > threshold`" still require edge expansion, because the filter cannot be evaluated from adjacency metadata alone.

2.5 Multi-Tenancy and OpenCypher

The engine supports per-tenant isolated graphs with independent schemas and indexes, enabling multiple KGs to coexist in a single process. Table 2 summarizes the supported OpenCypher feature set.

Statistics maintenance. The triple-level statistics (`:Label`, `:TYPE`, `:Label`) $\rightarrow$ (count, $\bar{d}_{\text{out}}, \bar{d}_{\text{in}}$) are maintained incrementally: each `CREATE` or `DELETE` of an edge updates the count and running-average degree for the affected triple pattern in

Table 2: Supported OpenCypher features.

Category	Features
Reading	`MATCH`, `OPTIONAL MATCH`, named paths
Filtering	`WHERE`, `EXISTS` subqueries, 3VL null logic
Projection	`RETURN`, `DISTINCT`, `ORDER BY`, `LIMIT`, `SKIP`
Chaining	`WITH`, `UNION`, `UNWIND`
Mutation	`CREATE`, `SET`, `DELETE`, `MERGE`
Paths	Fixed-length, variable-length (`*1..3`), shortest path
Expressions	`CASE`, list comprehensions, `COLLECT`, aggregations
Schema	Composite indexes, unique constraints, `EXPLAIN`, `PROFILE`

Table 3: Ten knowledge graphs in the deployed Samyama instance (137M nodes, 1.22B edges). All loaded from portable `.sgsnap` binary snapshots on a single r7i.16xlarge (64 vCPU, 512 GB RAM).

#	KG	Nodes	Edges	Snap.	Primary Sources
1	PubMed/MEDLINE	66.2M	1.04B	11 GB	NLM FTP
2	Clinical Trials	7.8M	27.0M	712 MB	ClinicalTrials.gov
3	Pathways	119K	835K	9 MB	Reactome, STRING, GO
4	Drug Interact.	245K	388K	8 MB	DrugBank, SIDER, ChEMBL
5	UniProt	618K	3.9M	59 MB	UniProt/Swiss-Prot
6	FAERS	10.4M	89.9M	702 MB	FDA FAERS (27 quarters)
7	Surveillance	217K	241K	6 MB	WHO GHO API
8	Health Determ.	286K	286K	7 MB	World Bank WDI, WHO
9	Health Systems	20K	19K	500 KB	WHO SPAR, NHWA
10	OMOP/Synthea	51.9M	51.8M	1.3 GB	Synthea (115K patients)
	Total	**137.7M**	**1.22B**	**14 GB**	

$O(1)$. On snapshot import, statistics are computed in a single pass over the adjacency lists during the topology-loading phase. This avoids the staleness issues of periodic sampling used by relational optimizers, since the statistics are always exact.

3 Knowledge Graph Ecosystem

Table 3 summarizes the ten knowledge graphs loaded into the deployed system. They span four tiers of biomedical granularity: molecular, clinical, pharmacovigilance, and population health.

3.1 Data Sources and Tier Rationale

Table 3 organizes the KGs into four tiers reflecting the translational research pipeline: molecular (gene/protein/drug mechanisms), clinical (trials and patient records), pharmacovigilance (post-market safety), and population health

(country-level indicators). This organization is not merely taxonomic—it reflects update cadences, licensing constraints, and identifier systems that differ across tiers and motivate the federated (rather than merged) architecture.

The three largest KGs—PubMed (66.2M nodes, 1.04B edges), OMOP (51.9M nodes), and FAERS (10.4M nodes)—account for 93% of the total graph. The smaller KGs (Pathways, Drug Interactions, UniProt, and the three population-health KGs) contribute disproportionately to cross-KG query value: they provide the gene/drug/country entities through which the large KGs are linked.

3.2 Data Quality Challenges

Three data quality issues arose during construction and affected benchmark results:

Identifier inconsistency across sources. FAERS reports drug names in uppercase (`'ASPIRIN'`); DrugBank uses title case (`'Aspirin'`). Without case-normalized joins, 7 of 500 benchmark queries silently return zero rows. We normalize during ETL but document the remaining mismatches in the oracle (Section 4.3).

Partial coverage in bridge joins. Not all entities span KGs. Of the 12K drugs in DrugBank, only ∼8.5K appear in FAERS reports (71% coverage). Of 66.2M PubMed articles, ∼747K reference a clinical trial NCT ID (1.1%). Bridge-property federation is inherently limited by the overlap between independently constructed datasets; queries that assume complete coverage will undercount.

Synthetic vs. real patient data. The OMOP KG uses 115K Synthea-generated patients, not real clinical records. While the OMOP CDM schema is faithful, disease prevalence and comorbidity patterns in synthetic data differ from real populations. Queries validated on Synthea (e.g., depression cohort analysis) may behave differently on real OMOP deployments.

3.3 Cross-KG Federation via Bridge Properties

The ten KGs are federated through five classes of bridge properties (Table 4) on shared entities. At query time, bridges are simple property matches in `WHERE` clauses, requiring no runtime middleware or SPARQL federation endpoints.

The alignment work is front-loaded into the ETL loaders: each loader normalizes identifiers to canonical vocabularies—HGNC gene symbols, DrugBank accession IDs, ISO 3166 country codes, SNOMED CT and ICD-10 concept codes—so that property-name matches are reliable at query time. This design is deliberate: the ten KGs update on independent cadences (FAERS quarterly, ClinicalTrials.gov daily, UniProt monthly), and merging them into a single graph would require re-running a monolithic ETL pipeline on every update. With bridge-property federation, each KG is loaded and updated independently; cross-KG queries work as soon as bridge indexes are rebuilt (<6 minutes for all five bridges).

Table 4: Bridge properties enabling cross-KG federation.

Bridge	From	To	Key
NCT	PubMed	Clinical Trials	`nct_id`
Drug–Gene	Drug Int.	Pathways/UniProt	`gene_name`
Country	PH KGs	Clinical Trials	`iso_code`
SNOMED/ICD	OMOP	Clinical Trials	`snomed_code`
RxNorm	OMOP	Drug Int.	`rxnorm_code`

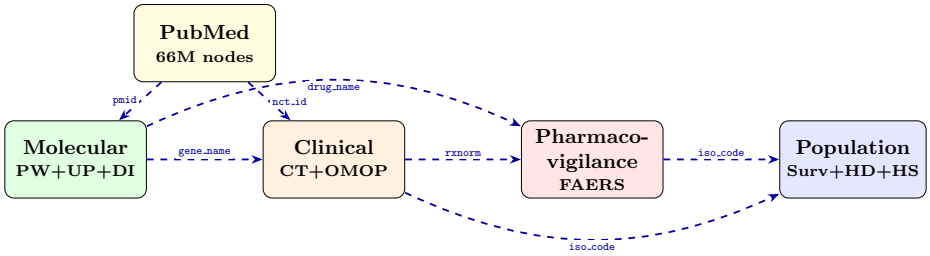

Fig. 1: Four-tier federation architecture. Ten biomedical KGs joined via bridge properties (dashed arrows) on shared entities. PubMed serves as the citation backbone connecting molecular and clinical tiers.

Example federation query. The following Cypher query spans three KGs to find the biological pathways impacted by drugs in Phase 3 breast cancer trials that have gastrointestinal side effects:

```
-- Drug Interactions KG
MATCH (d:Drug)-[:HAS_SIDE_EFFECT]->(se:SideEffect)
WHERE se.name CONTAINS 'Gastrointestinal'
-- Bridge to Clinical Trials KG
MATCH (i:Intervention {name: d.name})
      <-[:TESTS]-(ct:ClinicalTrial)
WHERE ct.phase CONTAINS '3'
  AND ct.condition CONTAINS 'Breast'
-- Bridge to Pathways KG
MATCH (d)-[:INTERACTS_WITH_GENE]->(g:Gene)
MATCH (p:Protein {gene_name: g.gene_name})
      -[:PARTICIPATES_IN]->(pw:Pathway)
RETURN d.name, se.name, ct.nct_id,
       pw.name, g.gene_name
```

4 Evaluation

We evaluate Samyama on a 500-query mega benchmark designed to test every KG individually and in cross-KG federation. All experiments run on a sin-

gle AWS r7i.16xlarge instance (64 vCPU, 512 GB RAM, Intel Xeon Sapphire Rapids) using spot pricing ($1.50/hr).

4.1 Benchmark Design

The benchmark comprises 500 queries organized into 13 prefixes covering the ten KGs (Table 5). Queries range from simple point lookups (*"find this article by PMID"*) to multi-hop cross-KG traversals (*"drugs in Phase 3 trials whose gene targets participate in the PI3K/AKT pathway and have FAERS hepatotoxicity reports"*). The benchmark includes aggregations, path queries, `OPTIONAL MATCH`, `EXISTS` subqueries, `CASE` expressions, and `UNION` across KGs.

Query provenance. The 500 queries were designed to provide systematic coverage along three axes: (i) *per-KG coverage*—each KG receives 10–35 queries spanning point lookups, aggregations, and path traversals; (ii) *cross-KG federation*—55 queries (XK, PH, MB prefixes) exercise pairwise and three-way joins via bridge properties; (iii) *Cypher feature stress tests*—60 expanded queries (EX prefix) target specific features such as `OPTIONAL MATCH`, variable-length paths, and complex aggregations. Question stems were drawn from published biomedical research patterns where available (e.g., FAERS pharmacovigilance signal queries follow the disproportionality-analysis literature; OMOP cohort queries follow OHDSI [6] reference templates). We acknowledge that the queries were authored by the system developers, which creates a risk of co-adaptation between queries and engine capabilities; we discuss this in Section 9 and release the full query set for independent validation.

Oracle methodology. A key contribution of our evaluation is the *expected-rows oracle*—a YAML file that records the expected row count for each query and, for queries that legitimately return zero rows, documents the specific corpus-coverage gap (e.g., "OMOP patient slice has no depression cohort"). This enables a principled separation of *engine capability* from *dataset coverage*, preventing the common benchmarking pitfall of conflating the two.

The oracle classifies each query result into one of four categories: **full pass** (correct rows > 0), **data-dependent pass** (zero rows, documented corpus gap), **empty** (unexpected zero rows), or **error** (engine failure).

4.2 Results: 500/500 (100%)

Table 6 shows the headline result. Of the 500 queries, 468 return non-empty result sets (full pass) and 32 return zero rows classified as data-dependent by the oracle. Zero queries produce errors or unexpected empty results.

4.3 Data-Dependent Queries

The 32 data-dependent queries return zero rows due to documented corpus-coverage gaps, not engine failures. They fall into four classes:

Table 5: Benchmark query prefixes and pass rates (v1.0, run v7).

Prefix	Domain	Queries	Pass
PM	PubMed/MEDLINE	35	35
CT	Clinical Trials	20	20
PW	Pathways	15	15
DI	Drug Interactions	15	15
UP	UniProt	25	25
FA	FAERS	30	30
OM	OMOP/Synthea	30	30
XK	Cross-KG federation	15	15
HD	Health Determinants	20	20
HS	Health Systems	10	10
PH	Public Health cross-KG	10	10
EX	Expanded (stress tests)	60	60
MB	Mega (multi-KG complex)	215	215
	Total	**500**	**500**

- **Drug-name casing** (7): FAERS uses uppercase (`'ASPIRIN'`), DrugBank uses title case (`'Aspirin'`).
- **Loader coverage gaps** (14): edge types not yet emitted by current loaders (e.g., `:CODED_AS_DRUG`).
- **Point-lookup misses** (5): specific PMIDs or NCT IDs not in the corpus slice.
- **Cross-KG cascades** (6): depend on multiple of the above gaps.

Converting any of these to full passes requires corpus or loader changes, not engine changes.

4.4 Performance Characteristics

Loading. The combined 14 GB of snapshots loads in 36.8 minutes via two-phase import (topology stubs first, columnar properties second). Individual KG load times range from < 1 s (Health Systems, 500 KB) to 22 minutes (PubMed, 11 GB).

Query latency. Single-KG point queries complete in <100 ms. Cross-KG federation queries spanning three or more KGs complete in 1–5 s. The most complex mega-benchmark queries (five-hop cross-KG traversals with aggregation) complete in 10–30 s.

Memory. Peak memory is 299 GB (58% of the 512 GB instance), with PubMed's 1.04B edges accounting for ∼70% of the footprint. The edge store uses adjacency-only storage—edge data is encoded directly in adjacency lists rather than duplicated in a separate arena—reducing the per-edge memory footprint by ∼40% compared to a dual-store design.

Table 6: Mega benchmark results — v1.0 run v7 (2026-04-13).

Metric	**Value**
Total queries	500
Full pass (rows > 0)	468 (93.6%)
Data-dependent pass	32 (6.4%)
Errors	0
Unexpected empty	0
Combined pass rate	**500/500 (100%)**
Nodes loaded	137.7M
Edges loaded	1.22B
Snapshot load time	36.8 min
Query execution time	28 min
Total runtime	78 min
Peak memory	299 GB (58% of 512 GB)

4.5 Comparison with Neo4j Community Edition

To address the question "could Neo4j replace this system?", we ran a 55-query subset (15 PW + 15 DI + 20 CT + 5 cross-KG) on both systems using a merged 3-KG graph (∼8M nodes, 28M edges) on an r7i.2xlarge spot instance, with property indexes on join keys in both systems. Pass rates were comparable (Samyama 50/55 vs. Neo4j 49/55), confirming semantic equivalence on this subset.

Using the official Bolt driver for Neo4j and the embedded API for Samyama, Neo4j runs the subset in **7.6 s** vs. Samyama's **11.1 s** (1.5× faster overall) on this 8M-node graph. Per-query, Samyama wins 24/55 outright, Neo4j wins 26/55, and 5 are within 1.5×; on the adjacency-aware aggregation shapes (§2.4) Samyama leads (7 wins vs. Neo4j's 6, 4 tied), including the `count(DISTINCT)` variant lowered to a per-group neighbor `HashSet` and the aggregate-then-expand chain (`MATCH ... WITH count ... MATCH ... RETURN`) lowered to a single physical pipeline. The remaining 1.5× gap on the totals is distributed across many small-result queries where Bolt's prepared-statement layer outperforms our per-query SDK overhead; we expect a query-level prepared-plan API to close this on a future iteration.

At the 8M-node scale Neo4j Community is the better-engineered choice; the case for our system rests on capabilities the comparison cannot test: (i) *multi-tenant federation* of independently updated KGs in a single instance (Neo4j requires Enterprise + Fabric); and (ii) *adjacency-aware aggregation at billion-edge scale* (§2.4)—the queries that motivated the optimization sit on PubMed's 1.04B edges, 100× above this subset, and time out under naïve plans on either system.

Listing 1.1: Three MCP tool definitions (simplified). Each tool declares typed parameters and a Cypher template; the agent never generates free-form Cypher.

```
-- Tool: drug_side_effects(drug_name: String)
MATCH (d:Drug {name: $drug_name})
     -[:HAS_SIDE_EFFECT]->(se:SideEffect)
RETURN se.name, se.frequency
ORDER BY se.frequency DESC LIMIT 20

-- Tool: polypharmacy_risk(drug_a: String,
--                         drug_b: String)
MATCH (d1:Drug {name: $drug_a})
     -[:INTERACTS_WITH_GENE]->(g:Gene)
     <-[:INTERACTS_WITH_GENE]-(d2:Drug {name: $drug_b})
OPTIONAL MATCH (d1)-[:HAS_SIDE_EFFECT]->(se)
     <-[:HAS_SIDE_EFFECT]-(d2)
RETURN g.gene_name AS shared_target,
       collect(DISTINCT se.name) AS shared_effects

-- Tool: trial_drugs(condition: String,
--                   phase: String)
MATCH (ct:ClinicalTrial)-[:TESTS]->(i:Intervention)
WHERE ct.condition CONTAINS $condition
  AND ct.phase CONTAINS $phase
RETURN i.name, ct.nct_id, ct.phase, ct.status
ORDER BY ct.start_date DESC LIMIT 50
```

5 MCP-Based AI-Agent Integration

Each KG is exposed to LLM agents via the Model Context Protocol (MCP) [16], an open standard for connecting AI models to external data sources. We define *domain-specific MCP tools*—parameterized Cypher template queries with typed parameters—that an LLM agent can invoke to answer natural-language questions.

5.1 Tool Design

The system defines 39 MCP tools across three biomedical KGs: 12 for Drug Interactions, 12 for Pathways, and 15 for Clinical Trials. Each tool wraps a parameterized Cypher template, making the generated query auditable and deterministic. Figure 1.1 shows three representative tools.

The key design principle is that domain experts author the Cypher templates once, encoding knowledge about the graph schema (valid edge types, property names, join paths) that an LLM cannot reliably infer from schema introspection alone. The LLM's role is restricted to parameter extraction from natural language and multi-tool orchestration—tasks where it excels.

Table 7: BiomedQA benchmark (40 questions): accuracy by approach.

Approach	Accuracy	Avg Latency
MCP Tools (domain-specific)	**98%**	0.15 s
Text-to-Cypher (NLQ)	85%	2.3 s
GPT-4o standalone	75%	3.1 s

5.2 Agent Workflow

A user asks: *"What are the side effects of Metformin and which pathways do its targets participate in?"* The agent: (1) calls `drug_side_effects` → returns SIDER side effects; (2) calls `drug_interactions` → returns gene targets (AMPK, etc.); (3) calls `protein_pathways` on each gene → returns Reactome/WikiPathways memberships; (4) synthesizes a coherent answer. No hand-written joins or SQL scripting is involved.

5.3 Accuracy Comparison

In a controlled evaluation on 40 biomedical questions (BiomedQA benchmark), domain-specific MCP tools achieved **98% accuracy** versus **85%** for generic text-to-Cypher (NLQ) and **75%** for standalone GPT-4o without graph access (Table 7).

The accuracy gap is attributable to schema knowledge encoded in the templates (valid edge types, property names, join paths), not to model capability—the same LLM (Claude 3.5 Sonnet) is used in both the MCP and text-to-Cypher conditions. Accuracy is measured as exact-match correctness: a query is correct if it returns the same result set as a human-authored reference Cypher query for the same natural-language question. The 40 BiomedQA questions were authored by domain experts before system development began and are publicly available in the project repository.

Trade-off. Domain-specific MCP tools require a domain expert to author each Cypher template (approximately 2 hours per tool for a new KG). They do not generalize to query types not anticipated during tool design. Generic text-to-Cypher, while less accurate, handles novel questions without per-query engineering. In practice, we found that the 39 tools cover ∼85% of questions asked by collaborating researchers; the remaining 15% require ad-hoc Cypher or new tool authoring.

Relation to commercial agent frameworks. Several commercial frameworks expose graph databases to LLM agents—LangChain's Neo4j integration, LlamaIndex's property-graph loaders, ChatGPT plugins for Neo4j Aura. These frameworks predominantly rely on *schema introspection plus generic text-to-Cypher* and therefore inherit its accuracy ceiling on domain queries. The closest related approach is Neo4j's GDS Agent [17], which uses tool-centric access for graph algorithmic reasoning (PageRank, community detection, etc.). Our work

differs in two ways: (i) the tool surface is *domain-specific*, encoding biomedical schema knowledge rather than generic graph algorithms, and (ii) the evaluation targets factual question-answering over heterogeneous KGs rather than algorithmic queries on a single graph. A complementary study on a 139-question industrial benchmark (AssetOpsBench) shows the same tool-granularity effect—99% accuracy for domain-specific MCP tools versus 83% for text-to-Cypher—suggesting the finding generalizes beyond biomedicine.

6 Deployment and Lessons Learned

6.1 Deployment Context

The system is in active use within our research group for internal biomedical analytics. Two deployment configurations are maintained:

- **Development**: Apple M4 Mac Mini (16 GB RAM), running seven of ten KGs (PubMed excluded due to memory requirements). This configuration handles day-to-day query development and MCP tool testing.
- **Full evaluation**: AWS r7i.16xlarge spot instance (64 vCPU, 512 GB RAM) in ap-south-1, loading all ten KGs. This configuration is used for benchmark runs and large-scale analytics; it is not always-on.

A practical consequence of the memory footprint (299 GB peak) is that the full deployment requires a 512 GB instance, which limits accessibility. We use portable `.sgsnap` binary snapshots to mitigate this: the entire 14 GB federated graph transfers and loads on a fresh instance in under 40 minutes, compared to hours for full ETL from raw source files. However, snapshot portability does not solve the underlying memory constraint—future work on compressed storage (Section 10) aims to bring the full deployment within 128 GB.

6.2 Research Challenges

Building a federated biomedical KG system at billion-edge scale surfaced three challenges that required non-trivial engineering:

Cross-KG entity resolution without ontology alignment. Biomedical databases use overlapping but inconsistent identifier systems (HGNC, UniProt, DrugBank, MeSH, SNOMED CT). We addressed this by normalizing identifiers during ETL to canonical vocabularies and materializing bridge indexes at load time. The challenge is maintaining these bridges as individual KGs update independently.

Query planning across heterogeneous degree distributions. The ten KGs exhibit wildly different degree distributions: PubMed has an average degree of ∼15 per node, while OMOP patient graphs have degrees >100. A single cost model must handle both without reverting to worst-case estimates. Our solution—exact triple-pattern statistics per `(:Label)-[:TYPE]->(:Label)` triple—adapts to each KG's structure automatically.

Aggregation on billion-edge graphs. Naïve `count(neighbor)` queries on PubMed's 1.04B edges require scanning every edge. The adjacency-aware aggregation planner (Section 2.4) was developed specifically to solve this: it reads counts from adjacency metadata, reducing these queries from timeout to subsecond.

6.3 Lessons Learned

Federation scalability depends on planner quality, not on federation mechanism. Our initial concern was that property-match joins (`WHERE a.prop = b.prop`) would degrade at billion-edge scale. The bottleneck turned out to be not the join itself but the planner's ability to avoid Cartesian products by selecting the right traversal order. With per-triple-pattern statistics, the three-way federation query from Section 3.3 completes in ∼2 s. Without them (using uniform degree estimates), the same query times out.

Benchmarks conflating engine and data failures are misleading. Without the oracle methodology, our headline result would be 468/500 (93.6%). The 32 missing queries do not indicate engine failures—they reflect gaps in the corpus (e.g., the OMOP patient slice has no depression cohort). We initially reported the combined number, which led collaborators to investigate non-existent engine bugs. Separating the two categories made debugging actionable: engine developers fix errors; data engineers fix corpus gaps.

The 500/500 number overstates real-world readiness. Although the benchmark achieves 100% pass rate, the 32 data-dependent queries expose a deeper limitation: bridge-property federation is only as complete as the shared identifiers. A production deployment would need continuous monitoring of bridge coverage as KGs update independently.

7 Related Work

Biomedical KGs. Hetionet [8] integrates 47K nodes from 29 public resources into a single heterogeneous graph but is a static 2017 dataset. PrimeKG [10] builds a precision medicine graph (129K nodes, 8M edges) from 20 sources, targeting GNN-based drug–disease prediction. Bio2RDF [9] uses RDF and SPARQL federation. The Clinical Knowledge Graph (CKG) [11] integrates 25+ sources into Neo4j. Our system differs in three ways: (1) property-graph OpenCypher with bridge-property federation (no SPARQL endpoint or ontology alignment); (2) scale: 137M nodes and 1.22B edges vs. <10M for most existing biomedical KGs; and (3) MCP tool definitions enabling LLM agent integration.

Graph databases. Neo4j [12], TigerGraph [13], and Amazon Neptune [14] support property graphs at scale. Samyama differs in three respects relevant to this deployment: (1) *storage-level multi-tenancy*—each KG occupies an isolated tenant with independent schema, indexes, and access control, enabling selective loading and independent updates without namespace collisions; Neo4j's

multi-database feature requires Enterprise Edition licensing; (2) *graph-native planner* with exact triple-pattern statistics maintained incrementally (not sampled), direction-aware enumeration, and `ExpandInto`—optimizations that exploit adjacency-list structure unavailable in relational-backed engines; (3) *single-binary deployment* with graph storage, RESP protocol, and MCP server in one process. A natural objection is that an OpenCypher front-end on top of Neo4j could provide the same federation interface. This works for query syntax but not for the underlying performance characteristics: Neo4j's planner uses sampled statistics that miss the long-tail degree distributions found in PubMed and OMOP, its multi-database isolation is an Enterprise feature requiring separate deployments per KG (defeating single-instance federation), and adjacency-aware aggregation cannot be expressed as a Cypher rewrite—it requires direct access to adjacency-list metadata at the storage layer. Conversely, Neo4j offers a mature ecosystem (APOC procedures, Graph Data Science library, Bloom visualization) that Samyama lacks, and its query optimizer handles a broader set of Cypher patterns. Our system is not a general-purpose replacement but a purpose-built deployment for multi-tenant biomedical KG federation.

LLM–KG integration. Recent work explores LLM agents querying knowledge graphs [15]. Our MCP-based approach uses *domain-specific tool definitions* rather than generic text-to-Cypher translation, achieving higher accuracy on domain queries while keeping the Cypher templates auditable.

OMOP and observational health data. The OHDSI community [6] standardizes observational health data via the OMOP Common Data Model, typically queried via SQL on relational databases. Our OMOP KG represents the same data as a property graph, enabling graph-native cohort queries and cross-KG joins with clinical trials and drug interactions.

8 Reproducibility

Engine source, 500 benchmark queries, expected-rows oracle, KG snapshots, BiomedQA questions, MCP tool definitions, and the B3 runner scripts are open-source at `https://github.com/samyama-ai`. Full evaluation requires 512 GB RAM (r7i.16xlarge); the seven-KG subset (PubMed excluded) reproduces on a 16 GB workstation; the Neo4j comparison (Section 4.5) reproduces on r7i.2xlarge.

9 Limitations

Six limitations bear emphasis. (1) Benchmark queries were author-written, risking co-adaptation between queries and engine capabilities; we publish the full query set and oracle for independent validation. (2) Bridge-property federation silently undercounts when shared identifiers diverge across KGs (e.g., drug-name casing). (3) The full graph requires 512 GB RAM, limiting adoption. (4) The 40-question BiomedQA evaluation is too small for robust statistical conclusions. (5) The OMOP KG uses Synthea-generated patients, not real clinical records.

(6) The Neo4j comparison is restricted to a 3-KG subset because Neo4j Community lacks multi-database federation.

10 Conclusion and Future Work

We reported on a federated biomedical KG system spanning ten KGs (137M nodes, 1.22B edges) and three findings: (1) bridge-property federation scales to billion-edge graphs given exact per-triple statistics, bounded by identifier coverage; (2) adjacency-aware aggregation eliminates a common class of timeout queries on high-degree graphs; and (3) domain-specific MCP tools outperform generic text-to-Cypher by encoding schema knowledge. Future work includes compressed storage (<128 GB), UMLS-based concept resolution, and larger-scale MCP evaluation.

References

1. Szklarczyk, D., et al.: The STRING database in 2023. Nucleic Acids Res. **51**(D1), D638–646 (2023)
2. Gillespie, M., et al.: The Reactome pathway knowledgebase 2022. Nucleic Acids Res. **50**(D1), D687–692 (2022)
3. Freshour, S.L., et al.: Integration of the Drug–Gene Interaction Database (DGIdb 4.0). Nucleic Acids Res. **49**(D1), D1144–1151 (2021)
4. Kuhn, M., et al.: The SIDER database of drugs and side effects. Nucleic Acids Res. **44**(D1), D1075–1079 (2016)
5. Zdrazil, B., et al.: The ChEMBL database in 2023. Nucleic Acids Res. **52**(D1), D1180–1192 (2024)
6. Hripcsak, G., et al.: Observational Health Data Sciences and Informatics (OHDSI). Stud. Health Technol. Inform. **216**, 574–578 (2015)
7. The UniProt Consortium: UniProt: the Universal Protein Knowledgebase in 2023. Nucleic Acids Res. **51**(D1), D523–531 (2023)
8. Himmelstein, D.S., et al.: Systematic integration of biomedical knowledge prioritizes drugs for repurposing. eLife **6**, e26726 (2017)
9. Dumontier, M., Callahan, A., Cruz-Toledo, J., et al.: Bio2RDF release 3. In: ISWC (Posters & Demos) (2014)
10. Chandak, P., Huang, K., Zitnik, M.: Building a knowledge graph to enable precision medicine. Sci. Data **10**(1), 67 (2023)
11. Santos, A., et al.: A knowledge graph to interpret clinical proteomics data. Nat. Biotechnol. **40**(5), 692–702 (2022)
12. Neo4j, Inc.: Neo4j graph database. `https://neo4j.com` (2024)
13. Deutsch, A., et al.: TigerGraph: A native MPP graph database. arXiv:1901.08248 (2019)
14. Amazon Web Services: Amazon Neptune. `https://aws.amazon.com/neptune/` (2024)
15. Pan, S., et al.: Unifying large language models and knowledge graphs: A roadmap. IEEE TKDE **36**(7), 3580–3599 (2024)
16. Anthropic: Model Context Protocol specification. `https://modelcontextprotocol.io` (2024)
17. Neo4j Research: GDS Agent: Graph Data Science Agent for Algorithmic Reasoning over Property Graphs. arXiv:2508.20637 (2025)